Chapitre 0 — Avant tout | Partie E — La croix : le joueur | leçon 0.69.4 sur 153 | difficulté 0 / 10

0.69.4. La fonction sprite() — un lutin au pixel près

#include <sprite> : sprite(numero, x, y, tuile) place un lutin — une tuile qui bouge pixel par pixel, par-dessus le fond.

1. Ce qu'il faut comprendre

Ce qu'elle fait : un **lutin** est une tuile qui n'est pas sur la grille : il se place au **pixel près**, par-dessus le fond. `sprite(numero, x, y, tuile)` place le lutin `numero` (0 à 39) en (x, y).

La ligne à écrire : `#include <sprite>`.

Ses arguments : le numéro du lutin, x de 0 à 159, y de 0 à 143, la tuile ; un cinquième, facultatif, le retourne (`MIROIR_X`, `MIROIR_Y`).

Ce qu'elle coûte : environ **145 octets** la première fois : les 40 lutins rangés, et leur copie à chaque image.

Essaie : fais-le descendre : `y = y + 1;`.

> **On retrouve sprite dans 55 leçons**
>
> 0.70, 0.70.1, 0.70.2, 0.108, 43, 45, 46, 48, 49, 50, 51, 83, 85, 99, 100, 104, 106, 107, 108, 109, 110, 110.3, 110.6, 110.7, 110.12, 110.13, 110.14, 110.15, 110.16, 110.18, 110.20, 111, 112, 113, 113.2, 113.4, 122, 124, 125, 127, 128, 131, 132, 134, 136, 137, 140, 143, 152, 153, 153.7, 153.9, 2.35, 2.37, 2.39

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **lutin (sprite)** — Un dessin qui bouge au pixel près, par-dessus le décor : le héros, un ennemi. sprite() en place un de 8 × 8, sprite16() un de 16 × 16. La console en a 40.
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **pixel** — Un tout petit carré de couleur ; l'écran en a 160 de large et 144 de haut.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <sprite> : un lutin au pixel près ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <sprite> : un lutin au pixel près ---- »). La console l'ignore.

`// Le A avance d'un pixel par image, de gauche à droite.`	Un commentaire : une phrase pour celui qui lit (« Le A avance d'un pixel par image, de gauche à droite. »). La console l'ignore.
`#include <sprite>     // sprite() : un lutin, au pixel près`	Demande à la console « sprite » (ce que c'est : place un lutin de 8 × 8 au pixel près). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « sprite() : un lutin, au pixel près ».)
`#include <ALPHABET>`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer.
`uint8_t x = 0;`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`x = x + 1;                    // un pixel de plus à chaque image`	x augmente de 1. (Le commentaire le dit : « un pixel de plus à chaque image ».)
`sprite(0, x, 72, ALPHABET[0]);      // le lutin 0, en (x, 72)`	Appel de la fonction sprite de la console, avec les arguments 0, x, 72, ALPHABET[0]. Ce qu'elle fait : un lutin 8 × 8 au pixel près. Le 5e argument le retourne : MIROIR_X, MIROIR_Y, DERRIERE. Un NOMBRE de 2 ou plus en 5e argument est une TAILLE : le dessin agrandi d'autant (le dessin n'est écrit qu'une fois). sprite(0, 36, 60, ROND, 4). (Le commentaire le dit : « le lutin 0, en (x, 72) ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **224 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage            50 o  ← la console, à l'allumage  — la mise en route de la console
  main()               39 o  ← démarrage  — ta fonction
  VBlank                8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage        28 o  ← image()  — attend l'image suivante
  AttendreVBlank       16 o  ← démarrage  — attend que l'écran soit libre
  RangerLutins         11 o  ← démarrage  — range les 40 lutins hors de l'écran
  InstallerTransfert   15 o  ← démarrage  — installe la copie des lutins en mémoire rapide
  CopierTuiles         31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  Tuiles               16 o  ← CopierTuiles  — 1 lettre sur 43
  DonneesTransfert     10 o  ← InstallerTransfert  — la petite routine qui copie les lutins
Lettres gravées : A
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
```

```
  sprite() × 1    23 o sur place, dans main()
  image()  × 1     6 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <sprite> : un lutin au pixel près ----
// Le A avance d'un pixel par image, de gauche à droite.

#include <sprite>     // sprite() : un lutin, au pixel près
#include <ALPHABET>
```

2. UNE VARIABLE — X

```
uint8_t x = 0;
```

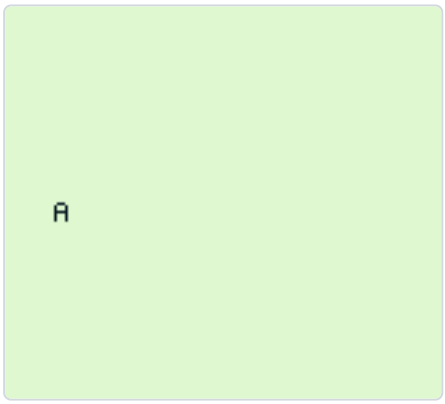
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();
    x = x + 1;                          // un pixel de plus à chaque image
    sprite(0, x, 72, ALPHABET[0]);      // le lutin 0, en (x, 72)
  }
}
```

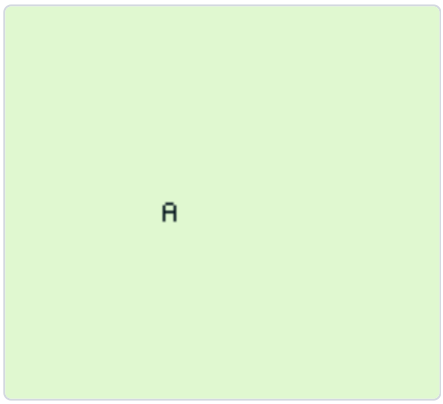
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

au bout de trois secondes — image 180

4. Ce qu’on doit voir

> Un A qui glisse doucement de gauche à droite, au milieu de l’écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ le lutin 0 porte le A
- ✓ il est à la hauteur 72
- ✓ il avance d’un pixel par image (+10)

Ensuite : **0.70. Plus fluide : la lettre au pixel près** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++